

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Scenario-Based Task (Medium)

Assessment Tool Weightage: 20%

QUESTIONS		
Q.No	Question	Bloom's Level
1	A university stores student and course data in a single table. Analyze the structure and identify normalization issues, then redesign it up to 3NF	. BL4 – Analyze
2	A company database contains employee and department details. Design appropriate SQL queries to retrieve employees working in multiple departments and justify your approach	. BL6 – Create
3	Given a relation with multiple functional dependencies, determine the candidate keys and check whether the relation satisfies BCNF. If not, decompose it.	BL5 – Evaluate
4	A banking system requires secure data access. Design views and constraints to ensure data security and integrity for different types of users.	BL6 – Create
5	A sales database contains redundant data leading to anomalies. Analyze the schema and apply normalization up to 3NF or BCNF to eliminate redundancy.	BL4 – Analyze
6	Given a dataset of customers and orders, write SQL queries using joins and subqueries to retrieve meaningful information such as frequent customers.	BL3 – Apply
7	A relation contains multi-valued dependencies. Analyze the structure and decompose it into 4NF, explaining the reasoning.	BL4 – Analyze

8	A database designer used improper constraints, leading to inconsistent data. Evaluate the schema and suggest appropriate integrity constraints.	BL5 – Evaluate
9	For a given relational schema, analyze the presence of join dependencies and decompose it into 5NF to remove redundancy.	BL6 – Create
10	A system uses inefficient SQL queries for data retrieval. Analyze the queries and optimize them using joins, indexing concepts, or query restructuring.	BL5 – Evaluate

Code of Conduct:

I VENNA MEENAKSHI REDDY (192471048) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: VENNA MEENAKSHI REDDY

RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding ; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1. A university stores student and course data in a single table. Analyze the structure and identify normalization issues, then redesign it up to 3NF

Given Single Table (Unnormalized Form - UNF):

Student_ID	Student_Name	Dept	Course_ID	Course_Name	Faculty_Name	Faculty_Phone
S101	Arun	CSE	C201	DBMS	Dr. Kumar	9876543210
S101	Arun	CSE	C202	OS	Dr. Ravi	9876501234
S102	Priya	ECE	C201	DBMS	Dr. Kumar	9876543210

Normalization Issues

1. Data Redundancy

- Student details are repeated for every course.
- Faculty details are repeated for every student taking the same course.

2. Update Anomaly

- If a faculty's phone number changes, it must be updated in multiple rows.

3. Insertion Anomaly

- A new course or faculty cannot be added until a student enrolls.

4. Deletion Anomaly

- If the last student enrolled in a course is deleted, course and faculty information is also lost.

First Normal Form (1NF)

Rule: Remove repeating groups and ensure atomic values.

Student_ID	Student_Name	Dept	Course_ID	Course_ Name	Faculty_Name	Faculty _Phone
S101	Arun	CSE	C201	DBMS	Dr. Kumar	9876543210
S101	Arun	CSE	C202	OS	Dr. Ravi	9876501234
S102	Priya	ECE	C201	DBMS	Dr. Kumar	9876543210

Primary Key: (Student_ID, Course_ID)

Second Normal Form (2NF)

Rule: Remove partial dependency.

Student Table

Student_ID (PK)	Student_Name	Dept
S101	Arun	CSE
S102	Priya	ECE

Course Table

Course_ID (PK)	Course_Name	Faculty_Name	Faculty_Phone
C201	DBMS	Dr. Kumar	9876543210
C202	OS	Dr. Ravi	9876501234

Enrollment Table

Student_ID (FK)	Course_ID (FK)
S101	C201
S101	C202

Third Normal Form (3NF)

Rule: Remove transitive dependency.

Student Table

Student_ID (PK)	Student_Name	Dept
S101	Arun	CSE
S102	Priya	ECE

Faculty Table

Faculty_ID (PK)	Faculty_Name	Faculty_Phone	Faculty_Phone
F101	Dr. Kumar	9876543210	9876543210
F102	Dr. Ravi	9876501234	9876501234

Course Table

Course_ID (PK)	Course_Name	Faculty_ID (FK)	Faculty_Phone
C201	DBMS	F101	9876543210
C202	OS	F102	9876501234

Enrollment Table

Student_ID (FK)	Course_ID (FK)
S101	C201
S101	C202
S102	C201

Final 3NF Schema

- Student (Student_ID, Student_Name, Dept)
- Faculty (Faculty_ID, Faculty_Name, Faculty_Phone)
- Course (Course_ID, Course_Name, Faculty_ID)
- Enrollment (Student_ID, Course_ID)

Advantages of 3NF

- Eliminates data redundancy.
- Prevents update, insertion, and deletion anomalies.
- Improves data consistency and integrity.
- Makes the database easier to maintain and query.

2. A company database contains employee and department details. Design appropriate SQL queries to retrieve employees working in multiple departments and justify your approach

Step 1: Table Creation:

```
MySQL 8.0 Command Line Cli  x  +  v

Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 20
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> create database dbms
-> ;
Query OK, 1 row affected (0.01 sec)

mysql> use dbms;
Database changed
mysql> CREATE TABLE Employee (
->     Emp_ID VARCHAR(10) PRIMARY KEY,
->     Emp_Name VARCHAR(50)
-> );
Query OK, 0 rows affected (0.04 sec)

mysql> CREATE TABLE Department (
->     Dept_ID VARCHAR(10) PRIMARY KEY,
->     Dept_Name VARCHAR(50)
-> );
Query OK, 0 rows affected (0.04 sec)

mysql> CREATE TABLE Employee_Department (
->     Emp_ID VARCHAR(10),
->     Dept_ID VARCHAR(10),
->     PRIMARY KEY (Emp_ID, Dept_ID),
->     FOREIGN KEY (Emp_ID) REFERENCES Employee(Emp_ID),
->     FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)
-> );
Query OK, 0 rows affected (0.09 sec)
```

Step 2: Data Insertion

```
MySQL 8.0 Command Line Cli  x  +  v  -  [icon]  x

mysql> INSERT INTO Employee VALUES
-> ('E101','John'),
-> ('E102','Alice'),
-> ('E103','David');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Department VALUES
-> ('D1','HR'),
-> ('D2','IT'),
-> ('D3','Finance');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Employee_Department VALUES
-> ('E101','D1'),
-> ('E101','D2'),
-> ('E102','D2'),
-> ('E103','D1'),
-> ('E103','D3');
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

Step 3: Display All Tables

```
mysql> SELECT * FROM Employee;
+-----+-----+
| Emp_ID | Emp_Name |
+-----+-----+
| E101   | John    |
| E102   | Alice   |
| E103   | David   |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Department;
+-----+-----+
| Dept_ID | Dept_Name |
+-----+-----+
| D1      | HR        |
| D2      | IT        |
| D3      | Finance   |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Employee_Department;
+-----+-----+
| Emp_ID | Dept_ID |
+-----+-----+
| E101   | D1      |
| E103   | D1      |
| E101   | D2      |
| E102   | D2      |
| E103   | D3      |
+-----+-----+
5 rows in set (0.00 sec)
```

Step 4: Retrieve Employees Working in Multiple Departments

```
mysql> SELECT e.Emp_ID, e.Emp_Name
-> FROM Employee e
-> JOIN Employee_Department ed
-> ON e.Emp_ID = ed.Emp_ID
-> GROUP BY e.Emp_ID, e.Emp_Name
-> HAVING COUNT(ed.Dept_ID) > 1;
+-----+-----+
| Emp_ID | Emp_Name |
+-----+-----+
| E101   | John    |
| E103   | David   |
+-----+-----+
2 rows in set (0.00 sec)

mysql> |
```

Justification

- The Employee_Department table stores the relationship between employees and departments.
- GROUP BY Emp_ID groups all department records for each employee.
- COUNT(Dept_ID) counts the number of departments assigned to each employee.
- HAVING COUNT(Dept_ID) > 1 filters only those employees who work in more than one department.
- This approach is efficient because it uses SQL aggregation instead of multiple joins or subqueries.

3. Given a relation with multiple functional dependencies, determine the candidate keys and check whether the relation satisfies BCNF. If not, decompose it.

Given Relation

R(Student_ID, Course_ID, Instructor, Department)

Functional Dependencies (FDs)

1. Student_ID, Course_ID $\rightarrow$ Instructor
2. Instructor $\rightarrow$ Department

Step 1: Find the Candidate Key

Compute the closure of {Student_ID, Course_ID}:

- (Student_ID, Course_ID) $\rightarrow$ Instructor
- Instructor $\rightarrow$ Department

So,

(Student_ID, Course_ID)⁺ = {Student_ID, Course_ID, Instructor, Department}

Hence,

Candidate Key = (Student_ID, Course_ID)

Step 2: Check BCNF

BCNF Rule:

For every functional dependency $X \rightarrow Y$, X must be a super key.

Check FD1

Student_ID, Course_ID $\rightarrow$ Instructor

- Left side is the candidate key.
- Satisfies BCNF.

Check FD2

Instructor $\rightarrow$ Department

- Instructor is not a candidate key.
- Therefore,
- Violates BCNF.

Hence, the relation is NOT in BCNF.

Step 3: BCNF Decomposition

Split the relation into two tables.

Table 1: Instructor

Instructor	Department
Dr. Kumar	CSE
Dr. Ravi	ECE

Table 2: Student_Course

Student_ID	Course_ID	Instructor
S101	C201	Dr. Kumar
S101	C202	Dr. Ravi
S102	C201	Dr. Kumar

Final BCNF Relations

Instructor

Instructor(Instructor, Department)

Primary Key: Instructor

Student_Course

Student_Course(Student_ID, Course_ID, Instructor)

Primary Key: (Student_ID, Course_ID)

Conclusion

- **Candidate Key:** (Student_ID, Course_ID)
- **BCNF Status:** Not satisfied because Instructor $\rightarrow$ Department violates BCNF.
- **BCNF Decomposition:**
 - Instructor(Instructor, Department)
 - Student_Course(Student_ID, Course_ID, Instructor)

4. A banking system requires secure data access. Design views and constraints to ensure data security and integrity for different types of users.

Step 1: Create the Account Table

Create an Account table to store customer information such as:

- Account Number
- Customer Name
- Balance
- Account Type
- Branch

Step 2: Insert Customer Data

Add sample customer account records into the Account table.

Step 3: Create a Customer View

Create a view that allows customers to access only their basic account information, such as:

- Account Number
- Customer Name
- Balance

This restricts access to confidential banking details.

Step 4: Create a Staff View

Create another view for bank staff that includes:

- Account Number
- Customer Name
- Balance
- Account Type
- Branch

This enables staff to perform banking operations while still controlling access to data.

Step 5: Apply Data Integrity Constraints

Add constraints to maintain accurate and valid data:

- **Primary Key:** Ensures each account number is unique.
- **Balance Constraint:** Prevents negative account balances.
- **Account Type Constraint:** Allows only valid account types such as Savings or

Current.

Step 6: Verify the Constraints

Attempt to add an account with an invalid balance (for example, a negative balance).

Result: The system rejects the record because it violates the balance constraint, ensuring data integrity.

OUTPUT:

```
MySQL 8.0 Command Line Cli x + v
Enter password: *****
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 21
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> use dbms
Database changed
mysql> CREATE TABLE Account (
->     Account_No INT PRIMARY KEY,
->     Customer_Name VARCHAR(50),
->     Balance DECIMAL(10,2),
->     Account_Type VARCHAR(20),
->     Branch VARCHAR(30)
-> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO Account VALUES
-> (1001, 'Arun', 25000.00, 'Savings', 'Chennai'),
-> (1002, 'Priya', 40000.00, 'Current', 'Coimbatore'),
-> (1003, 'Rahul', 15000.00, 'Savings', 'Madurai');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> CREATE VIEW Customer_View AS
-> SELECT Account_No, Customer_Name, Balance
-> FROM Account;
Query OK, 0 rows affected (0.01 sec)
```

```
MySQL 8.0 Command Line Cli x + v

mysql> SELECT * FROM Customer_View;
+-----+-----+-----+
| Account_No | Customer_Name | Balance |
+-----+-----+-----+
| 1001 | Arun | 25000.00 |
| 1002 | Priya | 40000.00 |
| 1003 | Rahul | 15000.00 |
+-----+-----+-----+
3 rows in set (0.01 sec)

mysql> CREATE VIEW Staff_View AS
-> SELECT Account_No, Customer_Name, Balance, Account_Type, Branch
-> FROM Account;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM Staff_View;
+-----+-----+-----+-----+-----+
| Account_No | Customer_Name | Balance | Account_Type | Branch |
+-----+-----+-----+-----+-----+
| 1001 | Arun | 25000.00 | Savings | Chennai |
| 1002 | Priya | 40000.00 | Current | Coimbatore |
| 1003 | Rahul | 15000.00 | Savings | Madurai |
+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> ALTER TABLE Account
-> ADD CONSTRAINT chk_balance
-> CHECK (Balance >= 0);
Query OK, 3 rows affected (0.15 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> ALTER TABLE Account
-> ADD CONSTRAINT chk_account_type
-> CHECK (Account_Type IN ('Savings', 'Current'));
Query OK, 3 rows affected (0.11 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Account VALUES
-> (1004, 'Kiran', -5000.00, 'Savings', 'Salem');
ERROR 3819 (HY000): Check constraint 'chk_balance' is violated.
mysql> |
```

Advantages

- Views restrict users from accessing unnecessary or sensitive data.
- PRIMARY KEY ensures each account is unique.
- CHECK constraints prevent invalid account types and negative balances.
- Improves data security and data integrity.

5. A sales database contains redundant data leading to anomalies. Analyze the schema and apply normalization up to 3NF or BCNF to eliminate redundancy.

Step 1: Original Table (Unnormalized Form)

Order_ID	Customer_ID	Customer_Name	Product_ID	Product_Name	Supplier_Name	Supplier_Phone
O101	C101	Arun	P101	Laptop	ABC Suppliers	9876543210
O101	C101	Arun	P102	Mouse	XYZ Suppliers	9876501234
O102	C102	Priya	P101	Laptop	ABC Suppliers	9876543210

Step 2: Analyze the Problems

Redundancy

- Customer details are repeated for every order.
- Supplier details are repeated for every product.

Anomalies

- **Update Anomaly:** Changing a supplier's phone number requires updating multiple rows.
- **Insertion Anomaly:** A new supplier cannot be added until a product is ordered.
- **Deletion Anomaly:** Deleting the last order for a product also removes supplier information.

Step 3: First Normal Form (1NF)

The table already satisfies 1NF because:

- All values are atomic.
- There are no repeating groups.

Step 4: Second Normal Form (2NF)

Remove partial dependencies.

Customer Table

Customer_ID	Customer_Name
C101	Arun
C102	Priya

Product Table

Product_ID	Product_Name	Supplier_Name	Supplier_Phone
P101	Laptop	ABC Suppliers	9876543210
P102	Mouse	XYZ Suppliers	9876501234

Orders Table

Order_ID	Customer_ID	Product_ID
O101	C101	P101
O101	C101	P102
O102	C102	P101

Step 5: Third Normal Form (3NF)

Remove transitive dependency.

Since:

Supplier_Name $\rightarrow$ Supplier_Phone

Create a separate Supplier table.

Supplier Table

Supplier_ID	Supplier_Name	Supplier_Phone
S101	ABC Suppliers	9876543210
S102	XYZ Suppliers	9876501234

Product Table

Product_ID	Product_Name	Supplier_ID
P101	Laptop	S101
P102	Mouse	S102

Customer Table

Customer_ID	Customer_Name
C101	Arun
C102	Priya

Orders Table

Order_ID	Customer_ID	Product_ID
O101	C101	P101
O101	C101	P102
O102	C102	P101

Final 3NF Schema

- Customer (Customer_ID, Customer_Name)
- Supplier (Supplier_ID, Supplier_Name, Supplier_Phone)
- Product (Product_ID, Product_Name, Supplier_ID)
- Orders (Order_ID, Customer_ID, Product_ID)

Conclusion

- The original sales database contained redundant customer and supplier information.
- The schema was normalized up to 3NF by separating customer, supplier, product, and order data into individual tables.
- This eliminates data redundancy and prevents update, insertion, and deletion anomalies, resulting in improved data consistency and integrity.

6. Given a dataset of customers and orders, write SQL queries using joins and subqueries to retrieve meaningful information such as frequent customers.

Step 1: Create Tables

```
MySQL 8.0 Command Line Cli x + v
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 23
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> use dbms
Database changed
mysql> CREATE TABLE Customer (
  ->   Customer_ID INT PRIMARY KEY,
  ->   Customer_Name VARCHAR(50),
  ->   City VARCHAR(30)
  -> );
Query OK, 0 rows affected (0.05 sec)

mysql> CREATE TABLE Orders (
  ->   Order_ID INT PRIMARY KEY,
  ->   Customer_ID INT,
  ->   Product VARCHAR(50),
  ->   Amount DECIMAL(10,2),
  ->   FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
  -> );
Query OK, 0 rows affected (0.06 sec)
```

Step 2: Data Insertion

```
MySQL 8.0 Command Line Cli x + v

mysql> INSERT INTO Customer VALUES
  ->   (101, 'Arun', 'Chennai'),
  ->   (102, 'Priya', 'Coimbatore'),
  ->   (103, 'Rahul', 'Madurai');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Orders VALUES
  ->   (1, 101, 'Laptop', 50000),
  ->   (2, 101, 'Mouse', 1000),
  ->   (3, 102, 'Keyboard', 1500),
  ->   (4, 101, 'Monitor', 12000),
  ->   (5, 103, 'Printer', 8000),
  ->   (6, 103, 'Scanner', 6000);
Query OK, 6 rows affected (0.01 sec)
Records: 6 Duplicates: 0 Warnings: 0
```

Step 3: Display All Tables

```
MySQL 8.0 Command Line Cli x + v

mysql> SELECT * FROM Customer;
+-----+-----+-----+
| Customer_ID | Customer_Name | City |
+-----+-----+-----+
|          101 | Arun          | Chennai |
|          102 | Priya         | Coimbatore |
|          103 | Rahul         | Madurai |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Orders;
+-----+-----+-----+-----+
| Order_ID | Customer_ID | Product | Amount |
+-----+-----+-----+-----+
|         1 |          101 | Laptop  | 50000.00 |
|         2 |          101 | Mouse   | 1000.00 |
|         3 |          102 | Keyboard | 1500.00 |
|         4 |          101 | Monitor | 12000.00 |
|         5 |          103 | Printer | 8000.00 |
|         6 |          103 | Scanner | 6000.00 |
+-----+-----+-----+-----+
6 rows in set (0.00 sec)
```

Step 4: Join Query

```
MySQL 8.0 Command Line Cli x + v

mysql> SELECT c.Customer_ID, c.Customer_Name, o.Product, o.Amount
-> FROM Customer c
-> JOIN Orders o
-> ON c.Customer_ID = o.Customer_ID;
+-----+-----+-----+-----+
| Customer_ID | Customer_Name | Product | Amount |
+-----+-----+-----+-----+
|          101 | Arun          | Laptop  | 50000.00 |
|          101 | Arun          | Mouse   | 1000.00 |
|          101 | Arun          | Monitor | 12000.00 |
|          102 | Priya         | Keyboard | 1500.00 |
|          103 | Rahul         | Printer | 8000.00 |
|          103 | Rahul         | Scanner | 6000.00 |
+-----+-----+-----+-----+
6 rows in set (0.00 sec)
```

Step 5: Find Frequent Customers (Using GROUP BY)

```
MySQL 8.0 Command Line Cli x + v

mysql> SELECT c.Customer_ID, c.Customer_Name, COUNT(o.Order_ID) AS Total_Orders
-> FROM Customer c
-> JOIN Orders o
-> ON c.Customer_ID = o.Customer_ID
-> GROUP BY c.Customer_ID, c.Customer_Name
-> HAVING COUNT(o.Order_ID) > 1;
+-----+-----+-----+
| Customer_ID | Customer_Name | Total_Orders |
+-----+-----+-----+
|          101 | Arun          | 3 |
|          103 | Rahul         | 2 |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

Step 6: Find Frequent Customers (Using a Subquery)

```
mysql> SELECT Customer_ID, Customer_Name
-> FROM Customer
-> WHERE Customer_ID IN
-> (
->     SELECT Customer_ID
->     FROM Orders
->     GROUP BY Customer_ID
->     HAVING COUNT(Order_ID) > 1
-> );
```

```
+-----+-----+
| Customer_ID | Customer_Name |
+-----+-----+
|          101 | Arun          |
|          103 | Rahul         |
+-----+-----+
```

```
2 rows in set (0.00 sec)
```

```
mysql> |
```

Conclusion

- JOIN combines data from the Customer and Orders tables.
- GROUP BY and HAVING identify customers with more than one order.
- The subquery retrieves the same frequent customers using a nested query. This is a complete MySQL solution suitable for university exams.

7. A relation contains multi-valued dependencies. Analyze the structure and decompose it into 4NF, explaining the reasoning.

Step 1: Original Relation

Assume the relation:

Student_ID	Hobby	Language
S101	Cricket	English
S101	Cricket	Tamil
S101	Music	English
S101	Music	Tamil
S102	Reading	Hindi
S102	Reading	English

Step 2: Analyze the Structure

Here:

- A student can have multiple hobbies.
- A student can know multiple languages.
- Hobbies and languages are independent of each other.

Multi-Valued Dependencies (MVDs)

- Student_ID $\twoheadrightarrow$ Hobby
- Student_ID $\twoheadrightarrow$ Language

These MVDs create redundant combinations of hobbies and languages.

Step 3: Check for 4NF

Rule of 4NF:

A relation is in Fourth Normal Form (4NF) if it contains no non-trivial multi-valued dependencies unless the determinant is a candidate key.

Since Student_ID $\twoheadrightarrow$ Hobby and Student_ID $\twoheadrightarrow$ Language are independent multi-valued dependencies, the relation is not in 4NF.

Step 4: Decompose into 4NF

Student_Hobby Table

Student_ID	Hobby
S101	Cricket
S101	Music
S102	Reading

Student_Language Table

Student_ID	Language
S101	English
S101	Tamil
S102	Hindi
S102	English

Final 4NF Schema

Student_Hobby

Student_Hobby(Student_ID, Hobby)

Primary Key: (Student_ID, Hobby)

Student_Language

Student_Language(Student_ID, Language)

Primary Key: (Student_ID, Language)

Reason for Decomposition

- The original relation had multi-valued dependencies, causing redundant combinations of hobbies and languages.
- Decomposing the relation into Student_Hobby and Student_Language removes these redundancies.
- The resulting relations satisfy Fourth Normal Form (4NF) because each table contains only one independent multi-valued attribute.

Conclusion

The original relation was not in 4NF due to the independent multi-valued dependencies:

- $\text{Student_ID} \twoheadrightarrow \text{Hobby}$
- $\text{Student_ID} \twoheadrightarrow \text{Language}$

After decomposition into Student_Hobby and Student_Language , redundancy is eliminated, and the schema satisfies 4NF.

8. A database designer used improper constraints, leading to inconsistent data. Evaluate the schema and suggest appropriate integrity constraints.

Step 1: Consider the Original Table

Employee

Emp_ID	Emp_Name	Age	Salary	Department
101	Arun	25	35000	HR
102	Priya	-5	40000	IT
101	Rahul	30	-10000	Sales
104	Kiran	28	30000	Finance

Step 2: Problems in the Schema

The table contains inconsistent data because of improper constraints:

- Duplicate Employee ID (Emp_ID = 101 appears twice).
- Negative Age (-5 is invalid).
- Negative Salary (-10000 is invalid).
- Department values should be restricted to valid departments.

Step 3: Appropriate Integrity Constraints

1. Primary Key Constraint

- Emp_ID should be the Primary Key.
- Ensures every employee has a unique ID.

2. NOT NULL Constraint

- Emp_Name and Department should not be empty.

3. CHECK Constraint

- Age must be greater than 18.
- Salary must be greater than 0.
- Department should allow only valid values such as HR, IT, Sales, or Finance.

4. UNIQUE Constraint

- Employee IDs must be unique to prevent duplicate records.

Step 4: Correct Data After Applying Constraints

Emp_ID	Emp_Name	Age	Salary	Department
101	Arun	25	35000	HR
102	Priya	26	40000	IT
103	Rahul	30	45000	Sales
104	Kiran	28	30000	Finance

Conclusion

By applying the following integrity constraints:

- PRIMARY KEY on Emp_ID
- NOT NULL on Emp_Name and Department
- CHECK on Age, Salary, and valid Department
- UNIQUE on Emp_ID

the database prevents duplicate records, invalid ages, negative salaries, and incorrect department values, thereby maintaining data consistency and integrity.

9. For a given relational schema, analyze the presence of join dependencies and decompose it into 5NF to remove redundancy.

Step 1: Original Relation

Supply(Supplier_ID, Part_ID, Project_ID)

Supplier_ID	Part_ID	Project_ID
S1	P1	J1
S1	P2	J1
S2	P1	J1
S2	P2	J2

Step 2: Analyze the Structure

The relation represents:

- A Supplier supplies a Part.
- A Supplier works on a Project.
- A Part is used in a Project.

These independent relationships create a Join Dependency (JD).

The relation can be reconstructed by joining smaller relations, indicating that it is not in Fifth Normal Form (5NF).

Step 3: Decompose into 5NF

Supplier_Part

Supplier_ID	Part_ID
S1	P1
S1	P2
S2	P1
S2	P2

Supplier_Project

Supplier_ID	Project_ID
S1	J1
S2	J1
S2	J2

Part_Project

Part_ID	Project_ID
P1	J1
P2	J1
P2	J2

Final 5NF Schema

Supplier_Part

Supplier_Part(Supplier_ID, Part_ID)

Primary Key: (Supplier_ID, Part_ID)

Supplier_Project

Supplier_Project(Supplier_ID, Project_ID)

Primary Key: (Supplier_ID, Project_ID)

Part_Project

Part_Project(Part_ID, Project_ID)

Primary Key: (**Part_ID, Project_ID**)

Reason for Decomposition

- The original relation contained a join dependency, causing redundant storage of data.
- By decomposing it into three smaller relations:
 - Supplier_Part
 - Supplier_Project
 - Part_Project
- The original relation can be reconstructed using natural joins without any loss of

information.

- This decomposition removes redundancy and satisfies Fifth Normal Form (5NF).

Conclusion

The original relation was not in 5NF because it contained join dependencies. After decomposing it into Supplier_Part, Supplier_Project, and Part_Project, redundancy is eliminated, and the schema satisfies Fifth Normal Form (5NF).

10. A system uses inefficient SQL queries for data retrieval. Analyze the queries and optimize them using joins, indexing concepts, or query restructuring.

Step 1: Create the Customer & Orders Tables

```
MySQL 8.0 Command Line Cli x + v
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 25
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> use dbms
Database changed
mysql> CREATE TABLE Customer (
->     Customer_ID INT PRIMARY KEY,
->     Customer_Name VARCHAR(50),
->     City VARCHAR(30)
-> );
Query OK, 0 rows affected (0.05 sec)

mysql> CREATE TABLE Orders (
->     Order_ID INT PRIMARY KEY,
->     Customer_ID INT,
->     Product VARCHAR(50),
->     Amount DECIMAL(10,2),
->     FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
-> );
Query OK, 0 rows affected (0.07 sec)
```

Step 2: Insert Sample Data

```
mysql> INSERT INTO Customer VALUES
-> (101, 'Arun', 'Chennai'),
-> (102, 'Priya', 'Coimbatore'),
-> (103, 'Rahul', 'Madurai');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Orders VALUES
-> (1, 101, 'Laptop', 50000),
-> (2, 101, 'Mouse', 1000),
-> (3, 102, 'Keyboard', 1500),
-> (4, 103, 'Printer', 8000);
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

Step 3: Inefficient Query

```
mysql> SELECT Customer_Name
-> FROM Customer
-> WHERE Customer_ID IN
-> (
->     SELECT Customer_ID
->     FROM Orders
-> );
+-----+
| Customer_Name |
+-----+
| Arun          |
| Priya         |
| Rahul        |
+-----+
3 rows in set (0.01 sec)
```

Step 4: Optimized Query Using JOIN

```
mysql> SELECT DISTINCT c.Customer_Name
-> FROM Customer c
-> JOIN Orders o
-> ON c.Customer_ID = o.Customer_ID;
+-----+
| Customer_Name |
+-----+
| Arun          |
| Priya         |
| Rahul        |
+-----+
3 rows in set (0.00 sec)
```

Step 5: Improve Performance Using an Index

```
mysql> CREATE INDEX idx_customer
-> ON Orders(Customer_ID);
Query OK, 0 rows affected (0.07 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> |
```

Optimization Techniques

1. Use JOIN instead of unnecessary subqueries for faster data retrieval.
2. Create indexes on frequently searched or joined columns (such as Customer_ID).
3. Select only the required columns instead of using SELECT *.
4. Use DISTINCT only when necessary to avoid extra processing.

Conclusion

The optimized query uses a JOIN instead of a subquery, making data retrieval more efficient. Creating an index on Customer_ID further improves performance by reducing the time required to search and join records. This results in faster and more efficient SQL query execution.